

Análisis del código: AR Hit Test con Three.js. Modelo con animación

1. IMPORTACIONES

```
import * as THREE from 'three';
```

```
import { ARButton } from 'three/addons/webxr/ARButton.js';
```

```
import { FBXLoader } from 'three/addons/loaders/FBXLoader.js';
```

<i>Instrucción</i>	<i>Qué hace</i>	<i>Analogía con Unity</i>
<code>`import * as THREE from 'three'`</code>	Importa toda la librería Three.js como Namespace `THREE`	<code>`using UnityEngine`</code>
<code>`import { ARButton }`</code>	Importa el botón prefabricado para Iniciar/detener AR	Similar a un UI Button con lógica de XR integrada
<code>`import { FBXLoader }`</code>	Importa el cargador de modelos FBX	<code>`AssetDatabase.LoadAssetAtPath`</code> o <code>`Resources.Load`</code>

2. DECLARACIÓN DE VARIABLES GLOBALES

```
let camera, scene, renderer;
```

```
let controller;
```

```
let reticle;
```

```
let lenador;
```

```
let mixer;
```

```
let hitTestSource = null;
```

```
const clock = new THREE.Clock();
```

Variable	Tipo	Función	Analogía Unity
`camera`	`PerspectiveCamera`	Cámara de la escena	`Camera.main`
`scene`	`Scene`	Contenedor de todos los objetos	`SceneManager.SetActiveScene()`
`renderer`	`WebGLRenderer`	Motor de renderizado	`Camera.Render()` interno
`controller`	`XRController`	Maneja input del controlador XR	`Input.GetButton` / `XRNode`
`reticle`	`Mesh`	Retícula visual de colocación	`GameObject` con sprite/mesh
`leñador`	`Group` (FBX)	Modelo animado del leñador	`GameObject` con `Animator`
		Reproduce animaciones del	
`mixer`	`AnimationMixer`	Modelo	`Animator` component
		Fuente de rayos para detectar	`ARRaycastManager` en
`hitTestSource`	`XRHitTestSource`	Superficies	AR Foundation
`clock`	`Clock`	Mide tiempo delta entre frames	`Time.deltaTime`

3. FUNCIÓN `init()` — CONFIGURACIÓN DE LA ESCENA

3.1 Contenedor DOM

```
const container = document.createElement('div');
document.body.appendChild(container);
```

Crea un `<div>` vacío en el HTML y lo agrega al `<body>`. El `renderer` de Three.js se adjuntará a este `div`.

3.2 Escena y Cámara

```
scene = new THREE.Scene();
camera = new THREE.PerspectiveCamera(70, window.innerWidth / window.innerHeight, 0.01, 20);
```

Parámetro	Valor	Significado
`70`	Field of View	Ángulo de visión vertical en grados
`window.innerWidth / window.innerHeight`	Aspect Ratio	Relación de aspecto de la pantalla
`0.01`	Near Clipping Plane	Objetos más cerca de 1cm no se Renderizan
`20`	Far Clipping Plane	Objetos más allá de 20m no se Renderizan

En AR, el `near plane` debe ser muy pequeño para que objetos cercanos no desaparezcan.

3.3 Iluminación

```
scene.add(new THREE.HemisphereLight(0xa5a5a5, 0x898989, 3));
const light = new THREE.DirectionalLight(0xffffff, 3);
light.position.set(1, 1, 1).normalize();
scene.add(light);
```

Tipo de luz	Función	Analogía Unity
`HemisphereLight`	Luz ambiental simulada (cielo + tierra)	`Ambient Light` + `Skybox`
`DirectionalLight`	Luz direccional paralela (sol)	`Directional Light`

Los colores 0xa5a5a5 (cielo) y 0x898989 (tierra) son grises cálidos. La intensidad 3 es alta porque la escena AR tiene fondo transparente y necesita luz suficiente.

3.4 Retícula

```
const reticleGeometry = new THREE.RingGeometry(0.05, 0.07, 32);
const reticleMaterial = new THREE.MeshBasicMaterial({
  color: 0x00ff00,
  side: THREE.DoubleSide,
  transparent: true,
  opacity: 0.8
});
reticle = new THREE.Mesh(reticleGeometry, reticleMaterial);
reticle.visible = false;
reticle.matrixAutoUpdate = false;
scene.add(reticle);
```

Instrucción

```
`RingGeometry(0.05, 0.07, 32)`
`MeshBasicMaterial`
`side: THREE.DoubleSide`
`transparent: true`
`opacity: 0.8`
`visible = false`
`matrixAutoUpdate = false`
```

Explicación

Anillo con radio interno 5cm, externo 7cm, 32 segmentos
 Material que no reacciona a luces (siempre se ve igual)
****Clave****: visible por ambas caras del polígono
 Habilita transparencia
 80% opaco, 20% transparente
 Inicia oculta hasta que el hit test detecte una superficie
****Crítico****: Three.js no sobrescribirá la matriz manual del hit test

3.5 Carga del modelo FBX

```
const loader = new FBXLoader();
loader.load('./models/fbx/Lenador.fbx', function (object) {
  lenador = object;
  lenador.scale.set(0.004, 0.004, 0.004);
  lenador.visible = false;
```

Instrucción

```
`FBXLoader()`  
`.load(url, onLoad)`  
`scale.set(0.004, 0.004, 0.004)`  
`visible = false`
```

Explicación

Crea instancia del cargador de archivos FBX
Carga asíncrona ejecuta `onLoad` cuando termina
Reduce el modelo a 0.4% de su tamaño original
Inicia oculto hasta el primer toque

La escala `0.004` es empírica: el modelo original probablemente está en centímetros o unidades grandes, y en AR las unidades son metros.

3.6 AnimationMixer

```
mixer = new THREE.AnimationMixer(lenador);  
if (lenador.animations && lenador.animations.length > 4) {  
  const action = mixer.clipAction(lenador.animations[4]);  
  action.play();  
}
```

Instrucción

```
`AnimationMixer(objeto)`  
`mixer.clipAction(clip)`  
`action.play()`
```

Explicación

Crea un mezclador de animaciones para ese Objeto
Crea una acción de reproducción para el clip
Inicia la reproducción del clip

Analogía Unity

`Animator` component
`Animator.Play("nombre")`
`Play()`

`lenador.animations[4]` accede al quinto clip de animación del FBX (índice basado en 0). El fallback a `[0]` asegura que si hay menos de 5 clips, al menos reproduzca el primero.

3.7 Configuración de sombras

```
lenador.traverse(function (child) {  
  if (child.isMesh) {  
    child.castShadow = true;  
    child.receiveShadow = true;  
  }  
});
```

Instrucción	Explicación
<code>`traverse(callback)`</code>	Recorre recursivamente todos los hijos del Objeto
<code>`child.isMesh`</code>	Verifica si el hijo es una malla
<code>`castShadow = true`</code>	La malla proyecta sombras
<code>`receiveShadow = true`</code>	La malla recibe sombras de otros Objetos
	<code>`GetComponentInChildren<MeshRenderer>()`</code>
	<code>`GetComponent<MeshFilter>() != null`</code>

Aunque configuramos sombras, en AR con fondo transparente (`alpha: true`) las sombras no son visibles a menos que haya un plano receptor debajo del modelo.

3.8 Renderer

```
renderer = new THREE.WebGLRenderer({ antialias: true, alpha: true });
renderer.setPixelRatio(window.devicePixelRatio);
renderer.setSize(window.innerWidth, window.innerHeight);
renderer.xr.enabled = true;
container.appendChild(renderer.domElement);
```

Parámetro/Instrucción	Explicación
<code>`antialias: true`</code>	Suaviza los bordes de los polígonos
<code>`alpha: true`</code>	**Crítico para AR** : fondo transparente, se ve la cámara real detrás
<code>`setPixelRatio(devicePixelRatio)`</code>	Usa la resolución nativa del dispositivo (evita blur en pantallas retina)
<code>`setSize(width, height)`</code>	Tamaño del canvas en píxeles
<code>`xr.enabled = true`</code>	**Habilita WebXR** : el renderer puede funcionar en modo AR/VR
<code>`renderer.domElement`</code>	El <code><canvas></code> HTML que Three.js crea y renderiza

3.9 Botón AR

```
document.body.appendChild(ARButton.createButton(renderer, {
  requiredFeatures: ['hit-test']
}));
```

Parámetro	Explicación
<code>`ARButton.createButton(renderer)`</code>	Crea un botón HTML con lógica integrada para Iniciar/detener AR
<code>`requiredFeatures: ['hit-test']`</code>	**Solicita al navegador** que la sesión AR incluya hit test

Sin `requiredFeatures: ['hit-test']`, el navegador podría iniciar AR sin permitir detectar superficies, y el código fallaría silenciosamente.

3.10 Controlador XR

```
controller = renderer.xr.getController(0);
controller.addEventListener('select', onSelect);
scene.add(controller);
```

Instrucción	Explicación
<code>`getController(0)`</code>	Obtiene el primer controlador XR (mano Derecha o pantalla táctil)
<code>`addEventListener('select', onSelect)`</code>	Escucha el evento "select" (toque En pantalla o click en controlador)
<code>`scene.add(controller)`</code>	Agrega el controlador a la escena (necesario para que el rayo del controlador funcione)

En AR móvil, el "controlador" es la pantalla táctil. El evento `select` se dispara al tocar la pantalla.

4. FUNCIÓN `onWindowResize()`

```
function onWindowResize() {
  camera.aspect = window.innerWidth / window.innerHeight;
  camera.updateProjectionMatrix();
  renderer.setSize(window.innerWidth, window.innerHeight);
}
```

Instrucción	Explicación
<code>`camera.aspect`</code>	Actualiza la relación de aspecto de la cámara
<code>`updateProjectionMatrix()`</code>	Recalcula la matriz de proyección con el nuevo aspecto
<code>`renderer.setSize()`</code>	Redimensiona el canvas al nuevo tamaño de ventana

Se ejecuta cuando el usuario rota el celular o cambia el tamaño de la ventana.

5. FUNCIÓN `onSelect()` — COLOCAR EL LEÑADOR

```
function onSelect() {
  if (reticle.visible && lenador) {
    const position = new THREE.Vector3();
    const quaternion = new THREE.Quaternion();
    const scale = new THREE.Vector3();

    reticle.matrix.decompose(position, quaternion, scale);

    lenador.position.copy(position);
    lenador.quaternion.copy(quaternion);
    lenador.visible = true;
  }
}
```

Instrucción	Explicación	Analogía Unity
<code>`reticle.visible`</code>	Verifica que la retícula esté activa (hay una superficie detectada)	<code>`gameObject.activeInHierarchy`</code>
<code>`new THREE.Vector3()`</code>	Crea vector 3D vacío	<code>`Vector3.zero`</code>
<code>`new THREE.Quaternion()`</code>	Crea cuaternión identidad	<code>`Quaternion.identity`</code>
<code>`matrix.decompose(pos, rot, scl)`</code>	**Extrae** posición, rotación y escala De una matriz 4x4	<code>`transform.GetPositionAndRotation()`</code>
<code>`position.copy(vector)`</code>	Copia valores de otro vector	<code>`transform.position = other.position`</code>
<code>`quaternion.copy(quat)`</code>	Copia valores de otro cuaternión	<code>`transform.rotation = other.rotation`</code>

Por qué decompose: La matriz del hit test es una matriz 4x4 que contiene posición, rotación y escala en un solo arreglo de 16 números. `decompose` la separa en componentes usables.

6. FUNCIÓN `animate()` — LOOP PRINCIPAL

```
renderer.setAnimationLoop(function (time, frame) {
```

Parámetro Qué contiene

``time`` Tiempo en milisegundos desde el inicio de la sesión XR
``frame`` ****Objeto `XRFrame`****: contiene toda la información del frame actual de WebXR

Crítico: `frame` solo existe cuando la sesión XR está activa. Si no hay sesión, `frame` es `undefined`.

6.1 Actualización de animación

```
const delta = clock.getDelta();
if (mixer) mixer.update(delta);
```

Instrucción	Explicación	
<code>`clock.getDelta()`</code>	Tiempo transcurrido desde el último frame en segundos	<code>`Time.deltaTime`</code>
<code>`mixer.update(delta)`</code>	Avanza la animación del modelo ese tiempo	<code>`Animator.Update()`</code>

6.2 Hit Test

```
if (frame) {  
  const referenceSpace = renderer.xr.getReferenceSpace();  
  const session = renderer.xr.getSession();
```

Instrucción	Explicación
<code>renderer.xr.getReferenceSpace()</code>	Obtiene el espacio de referencia "local" de la sesión XR
<code>renderer.xr.getSession()</code>	Obtiene el objeto <code>XRSession</code> nativo

6.2.1 Crear hit test source (una sola vez)

```
if (hitTestSource === null && session !== null) {  
  session.requestReferenceSpace('viewer').then(function (viewerSpace) {  
    session.requestHitTestSource({ space: viewerSpace }).then(function (source) {  
      hitTestSource = source;  
    });  
  });  
}
```

Instrucción	Explicación
<code>requestReferenceSpace('viewer')</code>	Pide el espacio de coordenadas del visor (cámara del usuario)
<code>requestHitTestSource({ space: viewerSpace })</code>	Crea un rayo que parte desde la posición del visor hacia adelante
<code>hitTestSource = source</code>	Guarda la fuente para usarla en cada frame

Se ejecuta solo una vez, cuando inicia la sesión AR. `hitTestSource` persiste hasta que termina la sesión.

6.2.2 Ejecutar hit test por frame

```
if (hitTestSource) {  
  const hitTestResults = frame.getHitTestResults(hitTestSource);
```

Instrucción	Explicación
<code>frame.getHitTestResults(source)</code>	Lanza el rayo y devuelve los puntos de impacto con superficies reales

6.2.3 Procesar resultados

```
if (hitTestResults.length > 0) {  
  const hit = hitTestResults[0];  
  const hitPose = hit.getPose(referenceSpace);  
  
  if (hitPose) {  
    reticle.visible = true;  
    reticle.matrix.fromArray(hitPose.transform.matrix);  
  }  
} else {  
  reticle.visible = false;  
}
```

Instrucción

`hitTestResults[0]`

`hit.getPose(referenceSpace)`

`hitPose.transform.matrix`

`matrix.fromArray(array)`

Explicación

Toma el primer (más cercano) punto de impacto

Convierte el impacto al espacio de referencia de la escena

Matriz 4x4 con posición + rotación + escala de la superficie

Convierte el arreglo de 16 números en matriz de Three.js

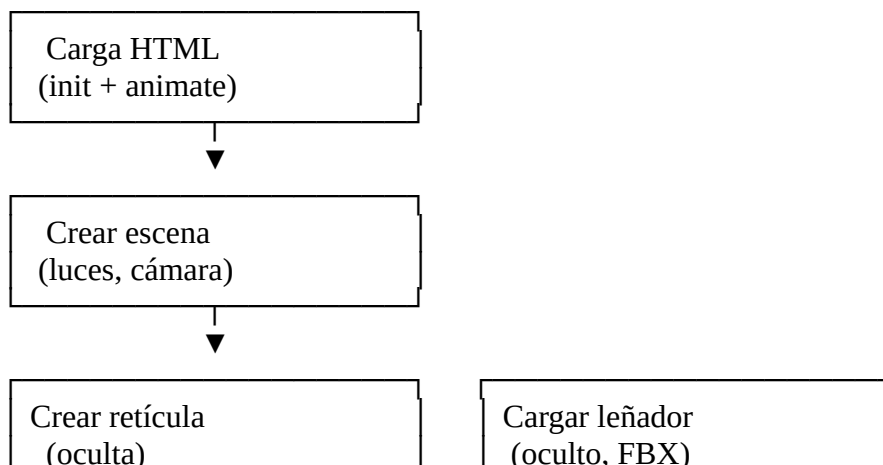
La matriz incluye la **orientación de la superficie**: si es pared vertical, la retícula se inclina verticalmente automáticamente.

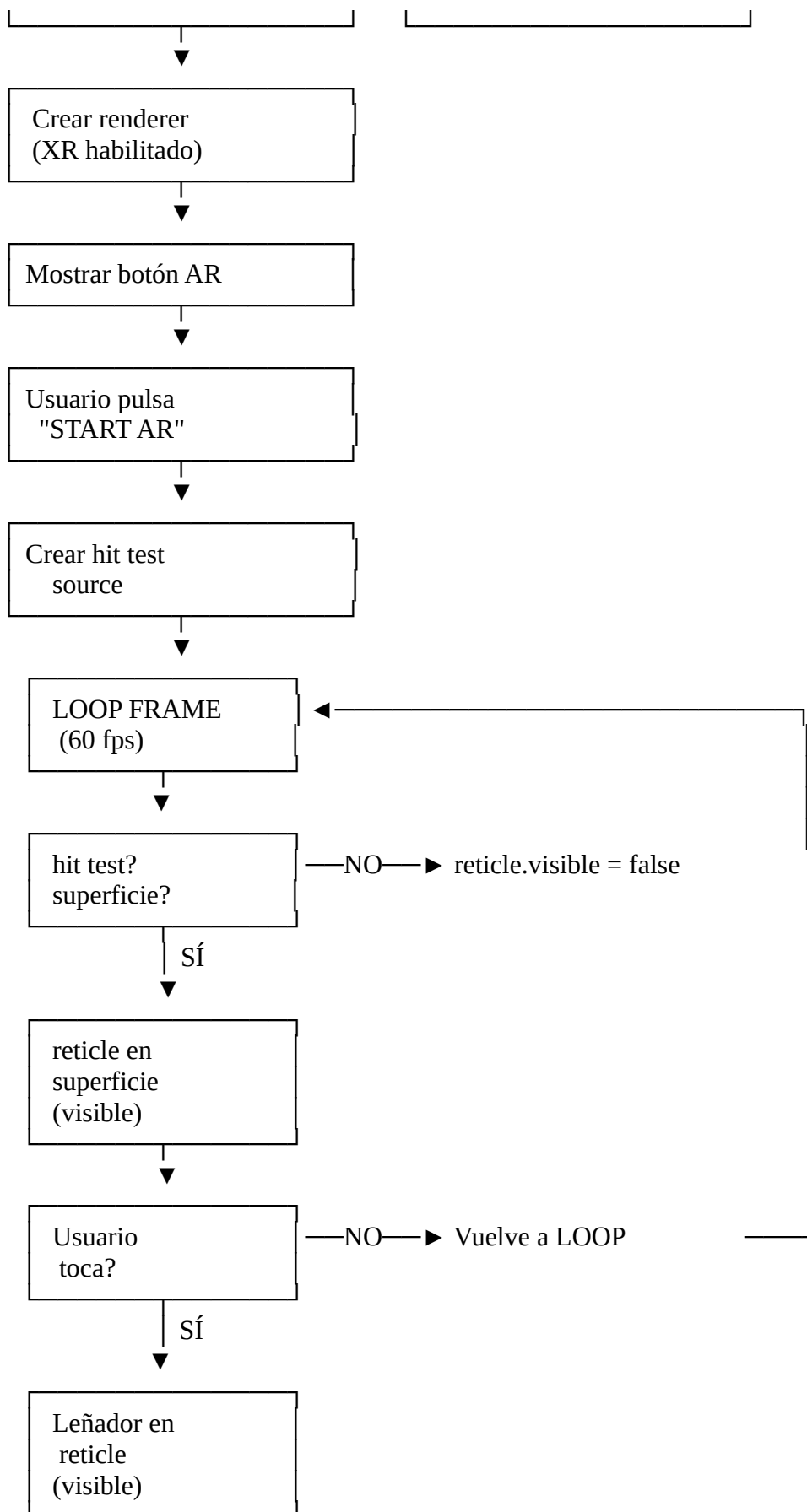
6.3 Renderizado

```
renderer.render(scene, camera);
```

Dibuja un frame completo: escena + cámara + luces + objetos.

Diagrama de flujo del programa





Puntos clave

Concepto

```
`alpha: true`  
`xr.enabled = true`  
`requiredFeatures: ['hit-test']`  
`matrixAutoUpdate = false`  
`frame` en  
`setAnimationLoop`  
`decompose`
```

Importancia

Sin esto no se ve la cámara real detrás
Sin esto no hay AR
Sin esto el navegador no permite detectar superficies
Sin esto la retícula ignora la matriz del hit test

Sin esto no hay acceso al hit test
Necesario para separar posición/rotación de la matriz 4x4

Código QR

